Convolutional Neural Networks — Part II

Dilated/Transposed Convolution, 효율적 합성곱, 구조적 입출력, CNN의 역사

2026-04-15

목차

1 한눈에 보기 **4**

2 복습: 합성곱의 기본 연산과 풀링 **5**
2.1 합성곱은 사실 교차상관(cross-correlation)이다 5
2.2 Stride: 다중률(multirate) 관점의 데시메이션 5
2.3 Padding: 경계 정보의 보존 6
2.4 출력 크기 공식 6
2.5 파라미터 수 세기 (CNN vs MLP) 6
2.6 풀링(Pooling): 풀에서 수를 고르는 연산 7
2.6.1 수용영역 점화식 8

3 Dilated (Atrous) Convolution **10**
3.1 정의와 유효 커널 크기 10
3.2 Stride와 무엇이 다른가 10

4 Transposed Convolution (Deconvolution) **12**
4.1 왜 “transposed”인가 — 선형사상의 전치 12
4.2 Stride가 있는 transposed conv와 용도 12
4.2.1 Fully Convolutional Network (FCN)과 의미분할 12

5 Vanilla CNN을 넘어서 1: 효율적 합성곱 **14**
5.1 Separable Convolution 14
5.2 Group Convolution 14

6 Vanilla CNN을 넘어서 2: 구조적 입출력과 입력 형태 **16**
6.1 구조적 출력 (Structured outputs) 16
6.2 다양한 입력 형태 (Input data types) 16

7 Vanilla CNN을 넘어서 3: Random / Unsupervised 필터 **17**

8 CNN의 신경과학적 기반 **18**

9 왜 CNN이 부흥했나: ImageNet과 짧은 역사 **19**
9.1 ImageNet — 대규모 이미지 데이터셋 19
9.2 LeNet → AlexNet → 더 깊게 19
9.3 What’s next? Transformer / ViT / gMLP 20

10 핵심 요약 **21**

11 부록: NumPy로 직접 확인하기 **22**
11.1 출력 크기·파라미터·수용영역 22
11.2 im2col 방식의 2D 교차상관 (dilated 포함) 22

11.3 Transposed convolution = 합성곱 행렬의 전치 23
11.4 Separable / Group / Pooling 23

1 한눈에 보기

이번 파트의 한 줄 요약: 합성곱(convolution)은 “2차원 정보를 파라미터 효율적으로 부호화하는 연산”이며, 이 부호화 방식을 늘이고(dilated), 뒤집고(transposed), 쪼개고(separable/group), 다른 입출력 구조로 확장하면서 현대 비전 모델로 이어진다.

아래 지도를 따라 진행한다.

```mermaid
flowchart TD
    A["복습: Conv / Stride / Padding / Pooling"] --> B["Dilated (atrous) Convolution"]
    A --> C["Transposed Convolution<br/>(deconvolution)"]
    C --> C1["Pixel-to-pixel 학습<br/>FCN / Semantic Segmentation"]
    A --> D["효율적 합성곱"]
    D --> D1["Separable Conv"]
    D --> D2["Group Conv"]
    A --> E["Vanilla CNN을 넘어서"]
    E --> E1["구조적 출력<br/>label / pixel / graph"]
    E --> E2["다양한 입력<br/>spectrogram / AlphaGo"]
    E --> E3["Random / Unsupervised filter"]
    A --> F["신경과학적 기반 (V1)"]
    A --> G["ImageNet과 CNN의 부흥<br/>LeNet→AlexNet→GoogLeNet→ResNet"]
    G --> H["What's next: Transformer / ViT / gMLP"]
```

> 이 노트의 사용법
>
> 각 절은 직접 정의 → 수학적 유도 → 기하/통계적 직관 → 무선통신·신호처리 비유의 순서로 읽도록 구성했다. 수식은 손으로 한 번 따라 쓰고, 마지막 섹션 11 의 NumPy 코드로 직접 돌려 확인하는 것을 권장한다.

---

2 복습: 합성곱의 기본 연산과 풀링

본격적인 확장에 앞서, 합성곱 계층을 정의하는 네 가지 양 — stride, padding, 출력 크기, 파라미터 수 — 과 풀링을 정밀하게 다시 정리한다. 이 부분은 시험에서 손계산을 요구하는 단골 영역이다.

2.1 합성곱은 사실 교차상관(cross-correlation)이다

딥러닝에서 "convolution"이라 부르는 연산은 신호처리의 진짜 합성곱(커널을 뒤집는)이 아니라 교차상관이다. 단일 채널에서 출력 (i, j) 성분은

$$y[i,j] \;=\; \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} x[\,i\,s+m,\; j\,s+n\,]\; w[m,n],$$

즉 입력 패치와 필터의 내적(inner product) 이다. 커널을 뒤집지 않으므로 학습되는 w 가 "이미 뒤집힌 커널"을 학습한다고 보면 되고, 결과적으로 학습 관점에서는 차이가 없다.

> **i** 내적 = 유사도라는 사실 (뒤에서 풀링 종류를 결정한다)
>
> $$\langle \mathrm{w}, \mathrm{x} \rangle = \|\mathrm{w}\|\,\|\mathrm{x}\| \cos\theta$$
>
> 필터 w 와 입력 패치 x 의 교차상관 값은 방향 정렬도(각거리) 를 잰다. $\theta \to 0$ 이면 값이 최대 → "이 위치에 이 패턴이 있다"는 강한 응답. 이 관점은 섹션 2.6 에서 max/min pooling 을 가르는 기준이 된다.

2.2 Stride: 다중률(multirate) 관점의 데시메이션

Stride s 는 필터를 s 픽셀씩 건너뛰며 적용하는 것이다. 인접한 응답들은 서로 매우 상관되어(중복) 있으므로, 고해상도(4K, 8K) 입력에서는 $s > 1$ 로 중복 계산을 줄이고 다운샘플링한다.

- 예: 7×7 입력 + 3×3 필터.
  - $s = 2 \to$ 한 축에 $\{0, 2, 4\}$ 위치에 깔끔히 맞아 3×3 출력.
  - $s = 3 \to \{0, 3\}$ 까지만 맞고 경계에서 정보가 잘려 나간다(아래 padding으로 보완).

2.3 Padding: 경계 정보의 보존

필터가 경계 밖으로 나갈 수 있도록 가장자리에 가상의 값을 덧대는 것. 세 가지가 흔하다.

방식	채우는 값	효과/비고
Zero padding	0	가장 흔함. 경계에서 곱해질 가중치를 사실상 "끄는" 효과 → 잡음 주입 없음. (값 범위 $[0, 255]$ 면 검은 테두리)
Replicate(repeat) padding	경계 픽셀 복제	경계값을 그대로 늘림 $(\dots, 0.5, 0.5 \mid \text{image} \mid 0.5, 0.5, \dots)$
Reflect(mirror) padding	경계 기준 거울 반사	$\dots, 1, 0.5 \mid \text{image} \mid 0.5, 1, \dots$ 처럼 대칭 반사

Zero padding은 FFT의 zero-padding / 선형 합성곱을 위한 경계 처리 와 정확히 같은 발상이다. 적절한 padding으로 출력 크기를 입력과 동일하게 맞출 수 있다("same" padding).

2.4 출력 크기 공식

$$\boxed{o = \left\lfloor \frac{i - k + 2p}{s} \right\rfloor + 1}$$

여기서 i 입력 크기, k 필터 크기, p 한쪽 padding, s stride. 라이브러리가 자동으로 해 주지만, 손계산이 시험에 나온다.

> ❗ 대표 예제 (반드시 손으로)
>
> 입력 32×32, 필터 5×5, stride 1, padding 2, 필터 10개.
>
> $$o = \frac{32 - 5 + 2 \cdot 2}{1} + 1 = 31 + 1 = 32.$$
>
> → 출력 텐서는 $32 \times 32 \times 10$ (필터 수 = 출력 채널 수).

2.5 파라미터 수 세기 (CNN vs MLP)

필터 하나의 파라미터 = (커널 부피) + (bias 1개). 5×5 필터를 3채널 입력에 적용하면

$$\underbrace{5 \times 5 \times 3}_{=75} + \underbrace{1}_{\text{bias}} = 76 \quad (\text{필터당}), \qquad 76 \times 10 \text{ filters} = \boxed{760}.$$

> **흔한 함정: 입력 채널 수는 필터 크기에 "자동으로" 따라온다**
>
> "5×5 필터"라고 말할 때 보통 공간 크기만 말한다. 입력이 3채널이면 필터는 자동으로 $5 \times 5 \times 3$ 이 된다. bias는 필터당 스칼라 1개($W * x + b$ 의 b). 그래서 250 이 아니라 760 이다.

같은 $32 \times 32 \times 3$ 입력을 MLP로 처리하면, 출력 뉴런 하나당 모든 픽셀과 연결되어

$$32 \times 32 \times 3 = 3072 \ (+1 \text{ bias} = 3073) \quad \text{(뉴런 1개당)}$$

의 가중치가 필요하다. 출력 뉴런이 많아지면 곱으로 폭증한다. 반면 합성곱은 필터 10개로 전체 위치를 공유하며 760개만 학습하고도 $32 \times 32 \times 10$ 특징맵을 만든다.

이 격차의 본질은 두 가지 귀납적 편향(inductive bias) 이다.

1. 가중치 공유(weight sharing): 같은 필터를 모든 위치에서 재사용 → 위치 불변 패턴을 자동 인코딩, 파라미터 급감.
2. 국소 연결(local connectivity): 패치 단위로만 본다 → "픽셀 간 관계를 일일이 학습"할 필요가 없어 데이터 효율 + 연산 효율 동시 확보.

학습 시 역전파는 한 위치가 아니라 필터가 슬라이딩한 모든 위치의 그래디언트를 합산하여 필터를 갱신한다(가중치 공유의 직접적 귀결):

$$\frac{\partial L}{\partial w[m,n]} = \sum_{i,j} \frac{\partial L}{\partial y[i,j]} \, x[\,is+m,\ js+n\,].$$

> **무선통신·신호처리 비유 — 매치드 필터와 LTI 시스템**
>
> 합성곱 계층은 입력 신호에 대한 선형 시불변(LTI) 필터 뱅크다. 각 필터는 특정 패턴에 대한 매치드 필터(matched filter) 로, 패치와의 상관(내적)이 클수록 큰 출력을 낸다 — 수신기에서 알려진 템플릿(파일럿/프리앰블)과의 상관이 SNR을 최대화하는 것과 동일한 원리다. 가중치 공유는 "탭 계수가 시간에 따라 변하지 않는다"는 시불변성 그 자체다.

2.6 풀링(Pooling): 풀에서 수를 고르는 연산

"Pooling"은 수들의 풀(pool)에서 하나의 수를 고르는 연산이다(영어 명사 pool → 동사 pooling). 필터를 여러 위치에 적용해 얻은 응답들의 풀에서 대표값 하나를 뽑는다.

종류	연산	언제 적합한가
Max pooling	$\max_k x_k$	필터 응답이 유사도(내적) 일 때. 가장 잘 맞은 위치만 남김
Average(mean) pooling	$\frac{1}{n}\sum_k x_k$	배경/전역 통계를 보존하고 싶을 때. 저역통과(평활화)
Min pooling	$\min_k x_k$	필터 응답이 거리(비유사도) 일 때. 가장 가까운(작은) 응답이 best match

⚠ 용어 주의: mean vs min

mean = 평균, min = 최소. 발음이 헷갈리지만 전혀 다르다. average pooling = mean pooling, 최솟값 풀링 = min pooling.

왜 종류가 갈리는가 (통계적·기하적 근거). 섹션 2.1 에서 보았듯 교차상관은 유사도 $\langle \mathrm{w}, \mathrm{x} \rangle = \|\mathrm{w}\|\|\mathrm{x}\| \cos\theta$ 를 잰다. 따라서 "가장 강한 패턴 일치"는 최댓값 → max pooling. 그러나 필터가 유클리드 거리 $\|\mathrm{w} - \mathrm{x}\|_2$ 같은 비유사도를 낸다면 best match는 최솟값 → min pooling. 연산의 성격이 풀링을 정한다.

통계적으로 보면, average pooling은 응답의 기댓값(저분산, 배경 쪽으로 편향) 을, max pooling은 극단값(고분산, 두드러진 활성 검출) 을 추정하는 것으로, 일종의 편향–분산 절충이다.

Max pooling이 주는 세 가지 이득과 한 가지 손해.

1. 수용영역(receptive field) 확대 — 파라미터 증가 없이. conv–pool을 쌓으면 같은 3×3 필터가 입력 좌표계에서 더 넓은 영역을 본다(섹션 2.6.1).
2. 중복/잡음 응답 제거. 인접 위치 응답은 강하게 상관(거의 중복)되므로 최댓값만 다음 층으로 전달 → 불필요한 잡음 억제, 연산 절약.
3. 국소 평행이동 불변성(translation invariance).
4. 손해: 정확한 위치 정보가 일부 소실된다. 분류엔 무방하나 위치가 중요한 과제(검출·분할)에선 average pooling이나 풀링 생략을 택한다.

이 "긍정 효과 ≫ 부정 효과"라는 균형 때문에 CNN뿐 아니라 MLP·Transformer 계열에서도 풀링이 널리 쓰인다.

2.6.1 수용영역 점화식

층 l 의 수용영역 r_l 과 "점프(jump)" J_l 은

$$r_l = r_{l-1} + (k_l - 1)\, J_{l-1}, \qquad J_l = J_{l-1}\, s_l, \qquad r_0 = 1,\ J_0 = 1.$$

구성	r (수용영역)
conv3(s1)	3
conv3(s1) → conv3(s1)	5
conv3(s1) → pool2(s2) → conv3(s1)	8
conv3 → pool2 → conv3 → pool2 → conv3	18

ℹ "풀링하면 넓게 본다"의 정확한 의미

직관적으로 "3×3 conv 뒤 2×2 풀링하면 다음 conv가 입력의 6×6 정도를 본다"고 말하지만, 겹침을 고려한 정확한 값은 점화식이 준다(위 표에서 8). 핵심은 풀링이 점프 J 를 곱셈적으로 키워 수용영역을 빠르게 확장시킨다는 점이다. 큰 필터를 쓰는 대신 풀링으로 수용영역을 키우면 파라미터·데이터 비용 없이 넓은 문맥을 얻는다.

무선통신·신호처리 비유 — 다중률 시스템의 데시메이션과 포락선 검출

풀링은 다중률(multirate) DSP의 데시메이션(decimation) 에 해당한다. stride/풀링으로 샘플레이트를 낮춰 중복을 제거한다. 특히 max pooling은 정류 후 포락선/피크 검출(envelope detection), average pooling은 저역통과 평균화 와 같다. “큰 필터 = 좁은 대역폭 선택성을 위한 긴 탭”인데, 풀링으로 해상도를 낮춘 뒤 같은 짧은 필터를 다시 쓰면 동일 탭 수로 더 넓은 주파수·공간 선택성을 얻는 셈이다.

---

3 Dilated (Atrous) Convolution

한 줄 정의: 필터 탭 사이에 구멍(hole)을 두어, 파라미터를 늘리지 않고 수용영역을 키우는 합성곱. atrous 는 불어 à trous("구멍이 있는")에서 왔다.

3.1 정의와 유효 커널 크기

확장률(dilation rate) d 에 대해 필터 탭을 d 칸 간격으로 배치한다. $k \times k$ 필터의 유효 크기는

$$k_{\text{eff}} = k + (k-1)(d-1) = d(k-1) + 1.$$

예: $k = 3,\ d = 2 \Rightarrow k_{\text{eff}} = 2 \cdot 2 + 1 = 5$. 즉 파라미터는 여전히 $3 \times 3 = 9$개지만, 입력 좌표계에서 5×5 영역을 덮는다. 출력 크기는

$$o = \left\lfloor \frac{i - k_{\text{eff}} + 2p}{s} \right\rfloor + 1.$$

3.2 Stride와 무엇이 다른가

```mermaid
flowchart LR
    subgraph Stride["Stride=2 (계산을 건너뜀)"]
      direction LR
      s["픽셀 일부는 합성곱 계산에서 제외됨"]
    end
    subgraph Dilated["Dilated rate=2 (계산은 모두 보되 성기게)"]
      direction LR
      di["모든 픽셀이 어느 출력엔가 기여, 단 탭은 성기게 샘플링"]
    end
```

- Stride: 출력 위치를 솎아내(데시메이션) → 일부 입력 픽셀이 어떤 출력에도 기여하지 못하고 버려질 수 있다.
- Dilated: 출력 해상도는 유지하되 탭을 성기게 둔다 → 픽셀을 통째로 버리지 않으면서 수용영역만 넓힌다.

따라서 "건너뛰고 싶지만(넓게 보고 싶지만) 어떤 픽셀도 통째로 빼고 싶지는 않을 때" dilated가 적합하다. 4K·위성영상처럼 인접 픽셀이 거의 동일한 기하정보를 담는 고해상도에서 특히 유용하다(큰 max pooling 또는 dilated 둘 중 택).

> 무선통신·신호처리 비유 — 희소 탭 FIR / 빗살(comb) 필터
>
> dilated conv는 탭 사이에 0을 끼운 희소 FIR 필터(zero-stuffed / sparse-tap filter)다. 다상(polyphase) 분해에서 업샘플된 필터, 혹은 주기적 응답을 갖는 빗살 필터(comb filter)와 같은 구조로, 탭(=파라미터) 수를 늘리지 않고 임펄스 응답의 지원(support)을 넓혀 공간 선택성을 키운다.

---

4 Transposed Convolution (Deconvolution)

한 줄 정의: 연산 자체는 합성곱과 동일하지만, "압축된 표현을 원래 크기로 펴는(upsampling)" 방향으로 쓰는 합성곱. 이름 deconvolution 은 오해를 부르며, 정확한 명칭은 transposed convolution 이다.

4.1 왜 "transposed"인가 — 선형사상의 전치

합성곱은 선형 연산이므로, 입력 벡터 x 에 대해 희소(Toeplitz) 행렬 C 로

$$\mathrm{y} = C\,\mathrm{x}, \qquad C \in \mathbb{R}^{(\text{작은 출력})\times(\text{큰 입력})}$$

로 쓸 수 있다. transposed convolution 은 같은 가중치로 만든 $C^\top$ 을 적용하여

$$\mathrm{z}' = C^\top \mathrm{z}, \qquad C^\top \in \mathbb{R}^{(\text{큰 입력})\times(\text{작은 출력})}$$

즉 작은(출력) 공간 → 큰(입력) 공간 으로 되돌린다. 순방향 conv의 그래디언트가 정확히 $C^\top$ 곱이라는 점에서, 이름이 자연스럽다. 정보 손실(예: max pooling)이 없다면 적절한 역가중치로 원 공간을 (이론상) 복원할 수 있다.

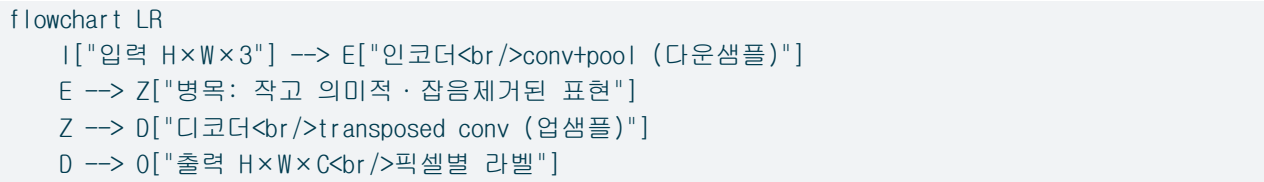
> **i** 수학적 보강 (선형대수)
>
> 선형사상 $T : \mathbb{R}^n \to \mathbb{R}^m$ 의 행렬이 C 이면, 그 수반(adjoint) T^* 의 행렬은 $C^\top$ 이다 ($\langle C\mathrm{x}, \mathrm{z}\rangle = \langle \mathrm{x}, C^\top \mathrm{z}\rangle$). transposed conv는 합성곱 연산자의 수반을 적용하는 것이며, 이는 신호처리의 분석-합성(analysis-synthesis) 쌍과 정확히 대응한다.

4.2 Stride가 있는 transposed conv와 용도

순방향에서 stride로 다운샘플한 만큼, transposed에서는 입력 사이에 0을 삽입(zero-insertion)한 뒤 합성곱하여 업샘플한다. 대표 용도는 pixel-to-pixel 학습이다.

4.2.1 Fully Convolutional Network (FCN)과 의미분할

목표가 "입력과 같은 크기의, 각 픽셀이 클래스 라벨을 갖는 의미맵(semantic map)"을 내는 것일 때:



- 같은 크기 conv를 끝까지 쌓아도 되지만, 중간에서 정보를 압축하지 못해 잡음을 끝까지 들고 가고, 넓은 문맥을 보려면 필터를 키워야 하는 부작용이 생긴다.
- 대신 인코더로 압축(작은 차원 + 풀링으로 잡음 제거) → 디코더로 transposed conv를 통해 원해상도로 복원하면, 중간 파라미터가 줄고 데이터 효율이 좋아진다. 이때 각 픽셀 위치는 더 이상 RGB가 아니라 의미(semantic) 값을 갖는다.

> 무선통신·신호처리 비유 — 보간 필터 / 합성 필터뱅크 / DAC 재구성
>
> transposed conv는 업샘플링(0 삽입) + 보간 필터 그 자체다. 분석–합성 필터뱅크의 합성(synthesis) 측, 또는 DAC의 재구성 필터에 대응한다. 인코더(분석)가 부대역(subband)으로 분해·압축하고, 디코더(합성)가 다시 합쳐 원신호 차원으로 복원하는 구조다.

---

5 Vanilla CNN을 넘어서 1: 효율적 합성곱

5.1 Separable Convolution

한 줄 정의: 2D 합성곱을 두 개의 1D 합성곱의 종속(cascade) 연산으로 분해해 곱셈 수를 줄이는 기법.

커널이 분리가능(separable, rank-1) 하면, 즉 $K = \mathrm{u}\,\mathrm{v}^{\top}$ 이면

$$K * x = \mathrm{u} *_{\mathrm{row}} (\mathrm{v} *_{\mathrm{col}} x).$$

출력 위치당 곱셈 수가 $k \times k$ 에서 $2k$ 로 준다($k = 3$ 이면 $9 \to 6$).

> ⚠ 왜 잘 안 쓰일까 — 병렬화 비용
>
> 모든 효율화에는 대가가 있다. separable은 종속 연산(먼저 1D conv → 그 출력에 다시 1D conv)이라, 코어가 9개 있어도 동시에 다 쓰지 못하고 첫 단계 출력을 기다려야 한다. 즉 연산량(FLOPs)은 줄지만 병렬화 효율이 나빠 벽시계 시간(wall-clock)에서 손해를 본다. GPU 시대에는 병렬성이 곧 속도라 잘 채택되지 않는다.

> ℹ 용어 구분 (정확성)
>
> 여기서의 separable은 공간적 분리(spatially separable): $k \times k$ 를 $k \times 1$ 과 $1 \times k$ 로 분해. MobileNet 등에서 흔한 깊이별 분리(depthwise separable) — 채널별 공간 conv + 1×1 pointwise conv — 와는 다른 개념이다. 후자가 실무에서 훨씬 널리 쓰인다.

5.2 Group Convolution

한 줄 정의: 입력 채널을 g개 그룹으로 나눠 그룹마다 독립적으로 합성곱한 뒤 이어붙이는(concat) 기법. 병렬화에 유리.

C_{in} 채널을 g 그룹으로 쪼개면, 각 필터는 C_{in}/g 채널만 보므로 커널 항 파라미터·FLOPs가 약 $1/g$ 로 준다. 그룹들이 서로 독립이라 여러 코어/GPU에 병렬 배치할 수 있다(AlexNet이 2-GPU 분할에 사용한 원조 사례).

$$\text{full: } (k^2 C_{\mathrm{in}} + 1) C_{\mathrm{out}} \xrightarrow{\ g \text{ groups}\ } \left(k^2 \tfrac{C_{\mathrm{in}}}{g} + 1\right) C_{\mathrm{out}}.$$

예: $k = 3,\ C_{\mathrm{in}} = C_{\mathrm{out}} = 64,\ g = 4 \Rightarrow 36{,}928 \to 9{,}280$ (약 $\times \tfrac{1}{4}$).

⚠ 대가 — 그룹 간 상호작용 손실

그룹을 나눈 만큼 채널 간 상호작용(cross-channel interaction) 이 끊긴다. 그룹 출력을 합칠 때 단순 평균/연결로는 정보가 일부 손실되어 separable과 달리 연산이 동치가 아니다. 이를 완화하려 채널 셔플(ShuffleNet)이나 interleaved group conv(Zhang et al., ICCV 2017)가 제안됐다.

	Separable conv	Group conv
분해 축	공간(spatial) 축 ($k \times 1$, $1 \times k$)	채널(channel) 축 (그룹 분할)
연산 동치성	동치 (정보 손실 없음)	비동치 (그룹 간 상호작용 손실)
병렬화	나쁨 (종속 연산)	좋음 (그룹 독립)
주 이득	FLOPs 감소	병렬화 + 파라미터 감소

무선통신·신호처리 비유 — 분리형 2D 필터 vs 부대역/안테나별 처리

separable conv는 영상처리의 분리형 2D 필터(행–열 분해, 분리형 FFT)와 동형이다. group conv는 MIMO에서 안테나/스트림별 병렬 처리나 OFDM의 부반송파 그룹별 처리처럼, 블록 대각(block-diagonal) 구조로 병렬 가지를 만드는 것에 대응한다(가지 간 결합을 끊는 대가로 병렬 이득).

6 Vanilla CNN을 넘어서 2: 구조적 입출력과 입력 형태

6.1 구조적 출력 (Structured outputs)

CNN의 출력은 과제에 따라 구조가 다르다.

출력 형태	과제	예
단일 라벨	분류	image classification
픽셀별 라벨	의미분할	FCN (섹션 4.2.1)
그래프(행렬)	그래프 예측	인접행렬로 표현되는 구조

그래프 출력은 인접행렬(adjacency matrix) 로 2D 표현된다. 단, 인접행렬의 행·열은 노드의 의미적 관계를 담으므로, 일반 3×3 conv로 다루면 그 구조 정보가 깨진다 → 그래프 신경망(GCN/GNN) 처럼 필터·출력 설계를 달리해야 한다.

6.2 다양한 입력 형태 (Input data types)

CNN은 "2D로 배열된 무엇이든" 받을 수 있다.

- 스펙트로그램(spectrogram): 음성·통신 신호의 시간–주파수 표현(STFT)을 2D 영상처럼 처리.
- AlphaGo 입력: 19×19 바둑판 격자에 17개 특징평면을 쌓은 $19 \times 19 \times 17$ 텐서. 평면들은 현재·과거 돌의 배치와 둘 차례 색을 부호화한다:

$$[X_t,\, Y_t,\, X_{t-1},\, Y_{t-1},\, \ldots,\, X_{t-7},\, Y_{t-7},\, C] \quad (8{+}\,8{+}\,1 = 17).$$

  이 텐서는 단일 "이미지"처럼 CNN으로 부호화된 뒤, MCTS·강화학습과 결합해 다음 수의 승률을 추정한다.

> 무선통신·신호처리 비유 — 시간–주파수 영상
>
> 스펙트로그램은 무선 신호 분석의 핵심 도구다. 변조 인식·간섭 검출·레이더 처리에서 STFT/스칼로그램을 2D 영상으로 보고 CNN을 적용하는 것은, 본질적으로 시간–주파수 평면에서 국소 패턴을 매치드 필터로 검출하는 일이다.

7 Vanilla CNN을 넘어서 3: Random / Unsupervised 필터

지금까지 필터는 지도학습으로 배웠지만, 다른 길도 있다.

- 무작위(random) 필터: 필터는 결국 입력을 부호화하는 기저(basis) 다. 충분히 많은 무작위 필터를 뽑으면 그중 서로 거의 직교한(uncorrelated) 부분집합이 포함되어, 학습된 필터에 버금가는 표현력을 보일 수 있다(Jarrett 2009; Saxe 2011; Pinto 2011; Cox & Pinto 2011). 학습 절차를 건너뛰는 대신, 동등 성능을 위해 보통 더 많은 필터(→ 추론 연산 증가)가 필요하다.
- 비지도(unsupervised) 학습 필터: 라벨 없이 필터를 배운다. 대표적으로 대조학습(contrastive learning) —
  - 같은 이미지에서 뽑은 패치 = 양성(positive),
  - 다른 이미지에서 뽑은 패치 = 음성(negative) — 으로 두고 표현을 학습한다(추후 CLIP/SigLIP 등으로 확장).

> 무선통신·신호처리 비유 — 무작위 투영 / 압축센싱
>
> 무작위 필터의 성공은 압축센싱(compressed sensing) 의 무작위 측정행렬(RIP 만족)과 같은 직관이다. 잘 설계된 무작위 기저는 신호의 정보를 거의 손실 없이 보존한다 — 무작위 부호(코드)나 무작위 투영이 효율적 부호화 수단이 되는 것과 동형이다.

---

8 CNN의 신경과학적 기반

CNN의 연산은 1차 시각피질(V1, primary visual cortex)의 처리와 닮았다.

- V1은 시각 신호를 처리하는 2D 구조를 가진다.
- 단순세포(simple cells) ≈ CNN의 선형 연산(곱·합).
- 복합세포(complex cells) ≈ CNN의 풀링 단위(비선형·국소 최대).
- CNN이 역전파로 학습한 1층 필터는 V1 메커니즘으로 알려진 Gabor 필터(방향성·국소 주파수 선택성, 등방성이 아님)와 닮은 모양으로 수렴한다.

그러나 인간 시각계와 CNN은 많이 다르다.

	시각 뇌(Visual brain)	CNN
해상도	낮음 (중심와 제외)	상대적으로 높음
감각 통합	다른 감각과 통합	순수 시각만
차원	풍부한 3D 정보	2D만
연산 메커니즘	알려지지 않음	곱셈·덧셈

(3D 지각의 증거: 한쪽 눈을 가리고 탁구를 치면 공의 깊이·타이밍을 못 잡는다 — 양안 시차로 3D를 재구성하기 때문.) Goodfellow 외, Deep Learning §9.10도 참고.

> 무선통신·신호처리 비유 — Gabor 원자와 시간-주파수 국소화
>
> Gabor 필터는 시간-주파수 분석의 Gabor 원자(가우시안 윈도우 × 정현파) 와 같다. 불확정성 한계에서 시간·주파수 국소화를 최적으로 절충하는 기저로, 국소 처프/톤 검출에 매치드 필터처럼 작동한다.

9 왜 CNN이 부흥했나: ImageNet과 짧은 역사

9.1 ImageNet — 대규모 이미지 데이터셋

- Fei-Fei Li(현 Stanford)가 Princeton 시절 2009년 구축. WordNet(Princeton의 단어 온톨로지 트리)에서 영감을 받아, 각 단어 노드에 해당하는 이미지를 모았다.
- 규모: 약 14M 이미지, 24k 범주. 검색으로 수집 후 크라우드소싱으로 노이즈를 정제(예: "apple"은 과일/회사가 섞임).
- 챌린지 부분집합(ImageNet-1K): 약 1.2M 이미지, 1000 범주. 대회를 열어 비전 연구를 폭발적으로 촉진했다.

> ℹ ImageNet 챌린지 정체기 (2009~2011)
>
> 초기 최고 알고리즘의 오류율은 ~ 35%(정확도 ~ 64%)에서 매년 1~3%p씩 더디게 개선되며 정체되는 듯 보였다. 라벨 노이즈 탓이라는 회의론도 있었다 — 곧 CNN이 이 판을 뒤집기 직전이었다.

9.2 LeNet → AlexNet → 더 깊게

```
timeline
    title CNN 아키텍처의 흐름
    1998 : LeNet (Yann LeCun) - 손글씨 숫자(MNIST) · 우편번호 인식
    2012 : AlexNet (Krizhevsky, Hinton) - ImageNet 오류율 급감, 부흥의 시작
    2014 : GoogLeNet (Inception) - 매우 깊은 망
    2015 : ResNet (Kaiming He) - 잔차연결로 초심층 학습
    2017 : DenseNet - 조밀 연결
    2021~ : ViT / gMLP - Transformer · MLP 계열로 전환
```

- LeNet (1998): LeCun이 손글씨 숫자 인식을 위해 제안. 5개 conv + 마지막 MLP 구조. MNIST로 숫자를 분류하고, 우편번호 인식에 활용. (이름은 LeCun의 Le에서.)
- AlexNet (2012): Krizhevsky가 Hinton(Toronto) 지도하에 LeNet과 유사한 7층(conv 5 + FC 2)을 구성, ImageNet 챌린지에서 오류율을 한 해 만에 약 10%p 떨어뜨림(기존 추세로는 5년 이상 걸릴 폭). 부흥의 신호탄.
- GoogLeNet (2014, Inception): 여러 크기 필터를 병렬로 묶은 Inception 모듈. 고성능 GPU 수 대에서 일주일 이상 학습. (보통 22층 깊이로 셈.)
- ResNet (2015, Microsoft Research Asia, Kaiming He): 잔차연결(skip connection) 로 152층 (실험적으로 1000층 이상)까지 안정적으로 학습. LeCun이 "ultra deep"이라 평.
- DenseNet (2017): 모든 앞 층을 뒤 층에 조밀하게 연결.

ℹ 잔차연결이 왜 깊은 망을 가능케 하나 (인과 사슬)

잔차블록 $y = x + f(x)$ 의 야코비안은 $\frac{\partial y}{\partial x} = I + \frac{\partial f}{\partial x}$. 항등항 I 때문에 역전파 시 그래디언트가 곱해져 사라지지 않고 보존된다 → 매우 깊은 망도 학습 가능 → 표현력·정확도 향상. (즉 항등 야코비안 항 → 그래디언트 보존 → 학습 가능성의 사슬.)

소프트웨어 생태계도 함께 폭발했다: DeCaf → Caffe(Yangqing Jia) → TensorFlow(Google) / Torch(LeCun) → PyTorch(현 PyTorch Foundation).

9.3 What's next? Transformer / ViT / gMLP

- Transformer (Vaswani et al., NeurIPS 2017): 현대 NLP의 표준 아키텍처. Attention Is All You Need.
- Vision Transformer (ViT) (Dosovitskiy et al., ICLR 2021): 이미지를 16×16 패치 토큰 열로 보고 Transformer로 처리. "An image is worth 16×16 words."
- gMLP (Liu et al., NeurIPS 2021): Pay Attention to MLPs — 게이트 MLP로 attention 없이도 경쟁력. "attention이 전부는 아닐 수 있다."

많은 2D 부호화 과제에서 이제 ViT가 CNN의 자리를 상당 부분 대체하고 있다.

---

10 핵심 요약

- CNN은 2D 정보를 파라미터·데이터 효율적으로 부호화하는 방법이며, 그 비결은 가중치 공유 + 국소 연결이라는 귀납적 편향이다.
- 구성요소: conv 계층 + pooling 계층 + MLP. 출력 크기 $o = \lfloor (i - k + 2p)/s \rfloor + 1$, 필터당 파라미터 $= k^2 C_{\text{in}} + 1$.
- Dilated: 파라미터 없이 수용영역 확대(성긴 탭). Transposed: 합성곱의 전치($C^\top$)로 업샘플(FCN/분할).
- 효율화: separable(공간 분해, 동치·병렬화 약함) / group(채널 분해, 비동치·병렬화 강함).
- CNN은 구조적 입출력(그래프→GNN), 다양한 입력(스펙트로그램, AlphaGo)으로 확장되고, 무작위/비지도 필터로도 학습 가능하며, V1과 닮았다.
- ImageNet + AlexNet이 부흥을 촉발 → GoogLeNet/ResNet/DenseNet → 지금은 ViT/gMLP로 무게추 이동.

연산	파라미터 변화	수용영역/해상도	핵심 키워드
Stride ↑	불변	해상도 ↓	데시메이션, 중복 제거
Pooling	0 (학습 X)	수용영역 ↑, 해상도 ↓	불변성, 잡음 제거
Dilated	불변	수용영역 ↑ (해상도 유지)	à trous, 희소 탭
Transposed	(필터만큼)	해상도 ↑	전치/수반, 업샘플
Separable	↓ (FLOPs)	불변	공간 분해, 종속
Group	↓ $(1/g)$	불변	채널 분해, 병렬

더 읽을거리

- Dive into Deep Learning (d2l.ai): 합성곱 기초(패딩·스트라이드·다중채널·풀링), 현대 CNN(AlexNet·VGG·NiN·GoogLeNet·BatchNorm·ResNet·DenseNet), 컴퓨터 비전 장의 전치합성곱·의미분할(FCN).
- Goodfellow, Bengio, Courville, Deep Learning, Ch. 9 (특히 §9.10 신경과학적 기반).
- G. Thomas, Mathematics for Machine Learning: 내적·노름·직교성, 선형사상과 전치(수반) — 섹션 2.6 의 유사도/거리와 섹션 4.1 의 전치 합성곱의 수학적 토대.

11 부록: NumPy로 직접 확인하기

아래 코드는 의존성 없이(numpy만) 그대로 실행된다. 개념을 손으로 확인하는 용도다.

11.1 출력 크기·파라미터·수용영역

```
import numpy as np

def out_size(i, k, s=1, p=0, d=1):
    """conv 출력 크기. d>1이면 dilated."""
    k_eff = d * (k - 1) + 1
    return (i - k_eff + 2 * p) // s + 1

def conv_params(kh, kw, c_in, c_out):
    """필터당 (kh*kw*c_in + 1), 총 *c_out."""
    return (kh * kw * c_in + 1) * c_out

def receptive_field(layers):
    """layers: [(k, s), ...]  반환: 입력 좌표계 수용영역."""
    r, J = 1, 1
    for (k, s) in layers:
        r = r + (k - 1) * J
        J = J * s
    return r

print(out_size(32, 5, s=1, p=2))                # 32
print(out_size(7, 3, s=2, p=0))                 # 3
print(out_size(7, 3, s=1, p=0, d=2))            # 3 (k_eff=5)
print(conv_params(5, 5, 3, 10))                 # 760
print(32 * 32 * 3 + 1)                          # 3073  (MLP 뉴런 1개)
print(receptive_field([(3,1),(2,2),(3,1)]))     # 8
```

11.2 im2col 방식의 2D 교차상관 (dilated 포함)

```
def conv2d(x, w, stride=1, pad=0, dilation=1):
    H, W = x.shape
```

```
    kh, kw = w.shape
    kh_e = dilation * (kh - 1) + 1
    kw_e = dilation * (kw - 1) + 1
    xp = np.pad(x, pad, mode='constant')
    Hp, Wp = xp.shape
    oh = (Hp - kh_e) // stride + 1
    ow = (Wp - kw_e) // stride + 1
    y = np.zeros((oh, ow))
    for i in range(oh):
        for j in range(ow):
            r0, c0 = i * stride, j * stride
            patch = xp[r0:r0+kh_e:dilation, c0:c0+kw_e:dilation]
            y[i, j] = np.sum(patch * w)        # 패치와 필터의 내적
    return y

x = np.arange(1, 26).reshape(5, 5).astype(float)
w = np.ones((3, 3))
print(conv2d(x, w).shape)                  # (3, 3)
print(conv2d(x, w, dilation=2).shape)      # (1, 1)
print(conv2d(x, w, stride=2, pad=1).shape) # (3, 3)
```

11.3 Transposed convolution = 합성곱 행렬의 전치

```
# 1D 예시: 순방향 conv를 행렬 C로 쓰고, 전치 C^T로 업샘플
x_in = np.array([1., 2., 3., 4.])
ker  = np.array([1., 0., -1.])
n_out = len(x_in) - len(ker) + 1
C = np.zeros((n_out, len(x_in)))
for i in range(n_out):
    C[i, i:i+len(ker)] = ker

y  = C @ x_in            # 순방향(다운): 4차원 -> 2차원
up = C.T @ np.array([1., 1.])   # 전치(업): 2차원 -> 4차원
print(y)                 # [-2. -2.]
print(up)                # [ 1.  1. -1. -1.]
```

11.4 Separable / Group / Pooling

```
# spatially separable: K = u v^T 이면 2D conv = (1D row) o (1D col)
u = np.array([1., 2., 1.]); v = np.array([1., 0., -1.])
K = np.outer(u, v)
```

```
full = conv2d(x, K)
sep  = conv2d(conv2d(x, u.reshape(3, 1)), v.reshape(1, 3))
print(np.max(np.abs(full - sep)))          # ~0.0 (동치)

# group conv 파라미터: g배 절감
def params(kh, kw, c_in, c_out, g=1):
    return (kh * kw * (c_in // g) + 1) * c_out
print(params(3, 3, 64, 64, g=1))           # 36928
print(params(3, 3, 64, 64, g=4))           # 9280

# max / average pooling
def pool(x, k=2, mode='max'):
    H, W = x.shape
    oh, ow = H // k, W // k
    out = np.zeros((oh, ow))
    for i in range(oh):
        for j in range(ow):
            blk = x[i*k:(i+1)*k, j*k:(j+1)*k]
            out[i, j] = blk.max() if mode == 'max' else blk.mean()
    return out

xp = np.arange(1, 17).reshape(4, 4).astype(float)
print(pool(xp, 2, 'max'))   # [[ 6.  8.] [14. 16.]]
print(pool(xp, 2, 'avg'))   # [[ 3.5  5.5] [11.5 13.5]]
```